

JuziHash: A 2048-bit Permutation-Based Hash
Function with Low-Cost Differential Security
Analysis

Hongjun Wu
Division of Mathematical Sciences
Nanyang Technological University, Singapore
`wuhongjun@gmail.com`

2026.06.30

Contents

1	Introduction	3
2	Definitions	4
2.1	Notations	4
2.2	Parameters	4
2.3	Operations	5
3	Functions	5
3.1	S-box	5
3.2	Linear transformation	7
3.3	Permutation $P_{2048}^{(r)}$	7
3.4	Round constants	9
3.5	Compression functions	9
3.5.1	The sponge update function f_{sp}	10
3.5.2	The full-state Davies–Meyer-like update function f_{DM}	10
4	JuziHash Algorithms	11
4.1	Padding the message	12
4.2	Setting the initial state $H^{(0)}$	12
4.3	Computing the final state $H^{(N)}$	13
4.4	Generating the message digest	13
4.4.1	JuziHash-512	14
4.4.2	JuziHash-1024	14
5	Security Analysis of JuziHash	14
5.1	JuziHash-512 sponge mode	14
5.2	JuziHash-1024 full-state Davies–Meyer-like mode	15
5.3	Free-start security	16
5.4	Extension security	17
5.5	Second-preimage and long-message preimage-type security	18
5.6	Herding attacks	19
5.7	Differential analysis of the 24-round permutation	20
5.8	Statistical randomness tests	22
5.9	Resisting cache-timing attacks	22
5.10	Expected generic security levels	23
6	Performance of JuziHash	24
7	Design Rationale	25
7.1	Rationale 1: low-cost differential analyzability	25
7.2	Rationale 2: simple four-dimensional AES-like structure	26
7.3	Rationale 3: shared hardware for the two main variants	27
7.4	Rationale 4: shared instruction-level acceleration	27

8	Conclusion	28
A	State Layout, Byte Permutations, and Message Embedding	28
A.1	State layout	29
A.2	S-box layer	29
A.3	MDS layer	30
A.4	Byte-permutation layer	30
A.5	Round-constant embedding	31
A.6	Message and chaining-value embeddings	31
B	Initial State Constants	32
B.1	JuziHash-512 initial state	32
B.2	JuziHash-1024 initial state	32
C	Bitsliced Implementation of the 4-bit S-box	32

1 Introduction

Cryptographic hash functions are usually evaluated in two different ways. The first one is the generic security of the mode, such as the birthday bound for collisions and the preimage bound implied by the digest length. The second one is the resistance of the underlying concrete primitive to structural attacks, especially differential attacks. The second aspect is often the harder one to make transparent. For example, the practical history of MD4, MD5, SHA-0 and SHA-1 shows that high-probability differential paths and local-collision techniques can be very difficult to exclude by informal reasoning alone [10, 23, 7, 5, 25]. SHA-2 and SHA-3 are mature standardized hash families [17, 18], but their differential analysis is not a small, easily reproducible calculation: SHA-1/SHA-2 use modular additions, Boolean functions, rotations and message expansion, while SHA-3 is based on a large bit-oriented permutation and is analyzed using techniques specialized to Keccak-like permutations [2]. These designs are highly important in practice, but they do not provide a simple byte-level active-S-box count that can be checked at low computational cost.

JuziHash is designed with a different emphasis: simplicity, regularity and low-cost differential analyzability. The core primitive is a 24-round 2048-bit permutation $P_{2048}^{(24)}$ built from an 8-bit S-box, length-4 MDS layers, byte permutations and round constants. The design is not an application of the AES block cipher, and it does not use AES as a black-box component. Rather, it extends a simple two-dimensional AES-like SPN design principle to a four-dimensional byte array. In this four-dimensional structure, the same byte S-box is applied independently to all bytes, the same length-4 MDS transformation is applied along state lines, and the byte-permutation layer changes the diffusion direction from round to round. Thus, the permutation is built from a small number of repeated and easy-to-specify operations instead of many unrelated round components.

This four-dimensional AES-like structure is chosen because its diffusion can be described and checked at the byte-activity level. Since all nonlinear operations are byte S-boxes and the linear layers have clear branch-number constraints, the minimum number of active S-boxes can be formulated as a compact MILP problem. In the current model, Gurobi proves that every 8-round differential trail has at least 200 active S-boxes, and the minimum-activity search completes in less than two minutes on a 24-core computer running at 2.4 GHz per core. This low-cost verification is a main design objective of JuziHash.

The 2048-bit permutation is used to define two hash variants. JuziHash-512 uses a sponge-like update function with a 2048-bit internal state and outputs a 512-bit digest. JuziHash-1024 uses the same permutation and the same message-injection hardware, but applies a full-state Davies–Meyer-like feed-forward update and outputs the 1024-bit capacity projection of the

final state. The second variant is introduced because a plain sponge using a 2048-bit permutation, a 1024-bit rate and a 1024-bit capacity gives only about 2^{512} generic preimage strength, whereas a 1024-bit digest naturally targets 2^{1024} generic preimage resistance. The full-state Davies–Meyer-like mode keeps the description close to a standard feed-forward structure, hides half of the final state against length extension, and allows both variants to share almost all implementation hardware.

The contribution of this specification is therefore not only a concrete hash-function proposal, but also a design methodology: start from a simple AES-like byte-oriented SPN principle, extend it regularly to four dimensions, and use the resulting large permutation so that the dominant differential-security claims can be checked automatically and cheaply. In this particular sense, JuziHash is intended to be more transparent than SHA-1, SHA-2 and SHA-3. This should not be interpreted as a claim that JuziHash uses AES, or that it has the same level of public cryptanalytic scrutiny or standardization maturity as the SHA families. Rather, the claim is that its differential diffusion can be analyzed by a much simpler and lower-cost computational model.

2 Definitions

2.1 Notations

The following notations are used in the JuziHash specification.

Word	A finite string of bits.
A^i	The i -th bit of the word A . For an m -bit word A , we write $A = A^0 \ A^1 \ \dots \ A^{m-1}$, where A^0 is the leftmost bit.
$\ $	Concatenation.
0^n	The all-zero string of length n bits.
$\lfloor x \rfloor$	The largest integer not greater than x .

2.2 Parameters

The following parameters are used in the JuziHash specification.

b	Size of the permutation state. In JuzziHash, $b = 2048$.
h	Size of the iterative chaining state. For both JuzziHash-512 and JuzziHash-1024, $h = 2048$.
d	Size of the message digest. For JuzziHash-512, $d = 512$. For JuzziHash-1024, $d = 1024$.
M	The message to be hashed.
ℓ	Length of M in bits. JuzziHash supports messages with $0 \leq \ell < 2^{64}$.
m	Message block size. In JuzziHash, $m = 1024$.
N	Number of 1024-bit blocks in the padded message.
$M^{(i)}$	The i -th 1024-bit message block, where $1 \leq i \leq N$.
$M^{(i),j}$	The j -th bit of $M^{(i)}$, where $M^{(i)} = M^{(i),0} \ M^{(i),1} \ \dots \ M^{(i),m-1}$.
$H^{(i)}$	The i -th 2048-bit chaining state. The value $H^{(0)}$ is the initial state, and $H^{(N)}$ is the final state before digest generation.
$H^{(i),j}$	The j -th bit of $H^{(i)}$, where $H^{(i)} = H^{(i),0} \ H^{(i),1} \ \dots \ H^{(i),h-1}$.
$P_{2048}^{(r)}$	The 2048-bit permutation with r rounds. Both JuzziHash variants use the 24-round permutation $P_{2048}^{(24)}$.
$RC(i)$	The 2048-bit round constant used before the four rounds $4i, 4i + 1, 4i + 2, 4i + 3$.

2.3 Operations

The following bitwise operations are used in the JuzziHash specification.

$\&$	Bitwise AND.
$ $	Bitwise OR.
$\oplus$	Bitwise XOR.
$\neg$	Bitwise complement.
$\text{rotl}_{64}(x, k)$	Left rotation of the 64-bit word x by k bits.

3 Functions

This section defines the S-box, the linear layer, the permutation, the round constants, and the two compression functions used in JuzziHash.

3.1 S-box

The 4-bit S-box S_4 used in JuzziHash is given in Table 1.

The 8-bit S-box S is constructed from S_4 by a three-round Feistel network. For an 8-bit input x , write

$$x = L_0 \| R_0,$$

Table 1: The 4-bit S-box S_4 .

x	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
$S_4(x)$	9	0	4	11	13	12	3	15	1	10	2	6	7	5	8	14

where L_0 and R_0 are 4-bit words. The three Feistel rounds are

$$\begin{aligned} L_1 &= R_0, & R_1 &= L_0 \oplus S_4(R_0), \\ L_2 &= R_1, & R_2 &= L_1 \oplus S_4(R_1), \\ L_3 &= R_2, & R_3 &= L_2 \oplus S_4(R_2). \end{aligned}$$

The output is

$$S(x) = L_3 \| R_3.$$

Implementation considerations. The 8-bit S-box has a simple and implementation-friendly structure. It is obtained from three Feistel rounds, each of which uses one evaluation of the 4-bit S-box S_4 and a few XOR operations.

The S-box can be implemented in constant time without secret-dependent memory accesses. In the table-shuffle AVX2 implementation, S_4 is evaluated using in-register table shuffles rather than memory table lookups indexed by secret data. More precisely, the 16-entry lookup table for S_4 is kept in vector registers, and the input nibbles select entries by vector shuffle instructions. Therefore, the S-box computation avoids cache-timing leakage caused by secret-dependent table indexing.

For bitsliced software and compact hardware implementations, the bit order is fixed as follows. For a 4-bit word

$$x = x_0 + 2x_1 + 4x_2 + 8x_3,$$

the bit x_0 is the least significant bit and x_3 is the most significant bit. The output is written in the same convention, $S_4(x) = y_0 + 2y_1 + 4y_2 + 8y_3$. A constant-time bitwise implementation of S_4 , using this least-significant-bit convention, is given in Appendix C.

This implementation is also efficient on SIMD platforms. Many independent S-box evaluations can be packed into vector registers and processed in parallel. The three-round Feistel structure then requires only three vector evaluations of S_4 , together with simple XOR and nibble manipulation operations.

The S-box is also suitable for hardware implementation. The component S_4 can be realized either as a small 16-entry combinational lookup table or as a compact Boolean circuit. For example, S_4 admits an implementation using only simple Boolean operations such as XOR, AND, ANDNOT, and OR. Hence the resulting 8-bit S-box has a simple structure for both software and hardware implementations.

3.2 Linear transformation

The linear layer uses the AES MixColumns matrix over $\mathbb{F}_{2^8}$. The field is represented as

$$\mathbb{F}_{2^8} = \mathbb{F}_2[z]/(z^8 + z^4 + z^3 + z + 1).$$

The MDS matrix is

$$A = \begin{pmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{pmatrix},$$

where the entries are hexadecimal representations of elements of $\mathbb{F}_{2^8}$. For

$$X = \begin{pmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{pmatrix} \in (\mathbb{F}_{2^8})^4,$$

the output is

$$Y = AX = \begin{pmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \end{pmatrix}.$$

All additions and multiplications in this layer are performed in $\mathbb{F}_{2^8}$. Since A is MDS, any nonzero input difference to this 4-byte transformation activates at least 5 input and output bytes in total.

3.3 Permutation $P_{2048}^{(r)}$

The permutation $P_{2048}^{(r)}$ is a 2048-bit permutation consisting of r rounds, where r is a multiple of 4. Conceptually, the permutation operates on a four-dimensional byte array

$$T[a][b][c][d], \quad 0 \leq a, b, c, d < 4.$$

Thus, the state contains

$$4 \times 4 \times 4 \times 4 = 256$$

bytes, or equivalently 2048 bits.

Each round consists of a nonlinear substitution layer and a linear diffusion layer. The substitution layer applies the 8-bit S-box to every byte of the state. In round i , the MDS layer is applied along dimension

$$(3 - i) \bmod 4.$$

That is, the active diffusion dimension changes from round to round, cycling through the four dimensions of the state array.

For implementation, the same 2048-bit state is stored as eight 256-bit state words

$$S = (S_0, S_1, \dots, S_7), \quad S_i \in \{0, 1\}^{256}.$$

Each state word consists of 32 bytes,

$$S_i = S_{i,0} \| S_{i,1} \| \dots \| S_{i,31}, \quad S_{i,j} \in \{0, 1\}^8.$$

The byte $S_{i,0}$ is the leftmost byte of S_i . The eight-word layout is used to describe the AVX2-oriented implementation; each S_i can be stored in one 256-bit AVX2 register. The exact correspondence between the four-dimensional array T and the eight state words $S_0, \dots, S_7$ is given in Appendix A.

Each round consists of four steps:

AddConstant, SubBytes, MDS, BytePerm.

The S-box layer applies the 8-bit S-box S independently to all 256 bytes of the four-dimensional state array. The MDS layer applies the 4×4 MDS matrix A to length-4 byte lines of the array. The byte-permutation layer then rotates byte positions so that bytes are moved along different dimensions in different rounds. These operations are equivalently described in the eight-word layout in Appendix A.

The round constants are injected once every four rounds, before the corresponding group of four rounds. The computation of $P_{2048}^{(r)}$ is shown in Algorithm 1.

Algorithm 1 The permutation $P_{2048}^{(r)}$

Require: A 2048-bit state $T[4][4][4][4]$

Require: Number of rounds r , where r is a multiple of 4

Ensure: Updated state T

```

1: for  $i = 0$  to  $r - 1$  do
2:   if  $i \bmod 4 = 0$  then
3:      $T \leftarrow T \oplus RC(i/4)$ 
4:   end if
5:    $T \leftarrow \text{SubBytes}(T)$ 
6:    $T \leftarrow \text{MDS}(T)$ 
7:    $T \leftarrow \text{BytePerm}_i(T)$ 
8: end for
9: return  $T$ 

```

3.4 Round constants

For each $i \geq 0$, the round constant $RC(i)$ is a 2048-bit string. Only its first 512 bits are nonzero, and the remaining 1536 bits are zero. We write

$$RC(i) = r(i) \| 0^{1536},$$

where $r(i)$ is 512 bits.

The initial value $r(0)$ is derived from π as

$$r(0) = \lfloor (\pi - 3)2^{512} \rfloor.$$

It is represented as eight 64-bit words

$$r(0) = r(0)_0 \| r(0)_1 \| \cdots \| r(0)_7,$$

where

$$\begin{aligned} r(0)_0 &= \text{0x243f6a8885a308d3}, \\ r(0)_1 &= \text{0x13198a2e03707344}, \\ r(0)_2 &= \text{0xa4093822299f31d0}, \\ r(0)_3 &= \text{0x082efa98ec4e6c89}, \\ r(0)_4 &= \text{0x452821e638d01377}, \\ r(0)_5 &= \text{0xbe5466cf34e90c6c}, \\ r(0)_6 &= \text{0xc0ac29b7c97c50dd}, \\ r(0)_7 &= \text{0x3f84d5b5b5470917}. \end{aligned}$$

For $i \geq 0$, each 64-bit word is updated independently by

$$r(i+1)_j = \text{rotl}_{64}(r(i)_j, 3) \oplus \text{rotl}_{64}(r(i)_j, 7) \oplus \text{rotl}_{64}(r(i)_j, 12), \quad 0 \leq j < 8.$$

3.5 Compression functions

JuziHash defines two iterative hash modes from the same 24-round 2048-bit permutation $P_{2048}^{(24)}$. Both modes process 1024-bit message blocks, and both introduce the message block into the 2048-bit state in exactly the same way: the message block is xored into four fixed state-word positions before the permutation is applied. This common message-injection path is useful for hardware implementations, because JuziHash-512 and JuziHash-1024 can share the same input-xor logic and the same 24-round permutation core.

When a 2048-bit state is represented as eight 256-bit state words

$$X = (S_0, S_1, \dots, S_7),$$

we define the 1024-bit rate projection and the 1024-bit capacity projection by

$$\text{Rate}_{1024}(X) = S_0 \| S_1 \| S_4 \| S_5,$$

$$\text{Cap}_{1024}(X) = S_2 \| S_3 \| S_6 \| S_7.$$

Thus, the rate positions are S_0, S_1, S_4, S_5 , while the capacity positions are S_2, S_3, S_6, S_7 . The 512-bit sponge output projection is

$$\text{Tr}_{512}^{\text{sp}}(X) = S_0 \| S_1.$$

JuziHash-512 outputs the first 512 bits of the rate part of the final sponge state. JuziHash-1024 outputs the 1024-bit capacity projection of its final 2048-bit state.

Let a 1024-bit message block be parsed as

$$M = M_0 \| M_1 \| M_2 \| M_3, \quad M_i \in \{0, 1\}^{256}.$$

The message block is embedded into the 2048-bit state by

$$\text{In}(M) = (M_0, M_1, 0^{256}, 0^{256}, M_2, M_3, 0^{256}, 0^{256}).$$

Equivalently, absorbing M means xoring M_0, M_1, M_2, M_3 into the state-word positions S_0, S_1, S_4, S_5 , respectively. This embedding is used by both JuziHash variants.

3.5.1 The sponge update function f_{sp}

The sponge update function is denoted by

$$f_{\text{sp}} : \{0, 1\}^{2048} \times \{0, 1\}^{1024} \rightarrow \{0, 1\}^{2048}.$$

For a 2048-bit state S and a 1024-bit message block M , it is defined by

$$f_{\text{sp}}(S, M) = P_{2048}^{(24)}(S \oplus \text{In}(M)).$$

This function first xors the message block into the rate positions and then applies the full 24-round permutation. JuziHash-512 uses f_{sp} directly as its sponge state update. Its internal state has rate 1024 bits and capacity 1024 bits. After all padded message blocks have been absorbed, the 512-bit digest is obtained by applying $\text{Tr}_{512}^{\text{sp}}$ to the final 2048-bit state.

3.5.2 The full-state Davies–Meyer-like update function f_{DM}

The second update function is denoted by

$$f_{\text{DM}} : \{0, 1\}^{2048} \times \{0, 1\}^{1024} \rightarrow \{0, 1\}^{2048}.$$

It is used for JuziHash-1024. It keeps the full 2048-bit state between message blocks, but applies a Davies–Meyer-like feed-forward xor to the 1024-bit capacity branch. Therefore the published 1024-bit digest is not the full state required to continue the iteration.

For a 2048-bit chaining state

$$S = (S_0, S_1, \dots, S_7),$$

first apply the same message-injection and permutation operation as in the sponge update:

$$Y = f_{\text{sp}}(S, M) = P_{2048}^{(24)}(S \oplus \text{In}(M)).$$

Write

$$Y = (Y_0, Y_1, \dots, Y_7).$$

The JuziHash-1024 update is

$$f_{\text{DM}}(S, M) = (Y_0, Y_1, S_2 \oplus Y_2, S_3 \oplus Y_3, Y_4, Y_5, S_6 \oplus Y_6, S_7 \oplus Y_7).$$

Equivalently,

$$\text{Rate}_{1024}(f_{\text{DM}}(S, M)) = \text{Rate}_{1024}(Y),$$

$$\text{Cap}_{1024}(f_{\text{DM}}(S, M)) = \text{Cap}_{1024}(S) \oplus \text{Cap}_{1024}(Y).$$

Thus, the rate branch of the permutation output is written back to the next 2048-bit state, while the capacity branch is xored with the previous capacity branch. The extra operation compared with the sponge update is only a 1024-bit feed-forward xor on the capacity branch.

This structure is still similar in spirit to a Davies–Meyer feed-forward construction, because the previous chaining state contributes linearly to part of the next state after a large nonlinear transformation. However, the primitive is a fixed public 2048-bit permutation rather than a block cipher keyed by the message. The important implementation point is that JuziHash-512 and JuziHash-1024 use the same input operation $S \leftarrow S \oplus \text{In}(M)$ and the same permutation core. The JuziHash-1024 mode only changes the write-back rule after the permutation.

Keeping the full 2048-bit state is also important for length-extension resistance. The JuziHash-1024 digest is the 1024-bit capacity projection of the final state, but the next update also needs the hidden 1024-bit rate projection. Hence a public digest is not a valid internal state for continuing the iteration.

Thus, JuziHash-512 uses f_{sp} directly, while JuziHash-1024 uses the full-state Davies–Meyer-like update function f_{DM} . Both variants are based on the same 24-round permutation $P_{2048}^{(24)}$ and the same 1024-bit message-injection path.

4 JuziHash Algorithms

JuziHash consists of four steps: padding the message M (Sect. 4.1), setting the initial state $H^{(0)}$ (Sect. 4.2), processing the padded message blocks to

compute $H^{(N)}$ (Sect. 4.3), and generating the message digest (Sect. 4.4). The two variants use the same 1024-bit message block size and the same permutation $P_{2048}^{(24)}$. Both variants keep a 2048-bit internal state. JuziHash-512 uses the sponge state update, while JuziHash-1024 uses the full-state Davies–Meyer-like update described above.

4.1 Padding the message

The message M is padded by zero extension followed by one final length block. Let the length of M be ℓ bits. JuziHash accepts messages with

$$0 \leq \ell < 2^{64}.$$

Although the final length field occupies one full 1024-bit block, the encoded value is a 64-bit message length. First append

$$z = (-\ell) \bmod 1024$$

zero bits, so that the length becomes a multiple of 1024 bits. Then append one additional 1024-bit block

$$\text{Len}_{1024}(\ell) = 0^{960} \parallel \text{enc}_{64}(\ell),$$

where $\text{enc}_{64}(\ell)$ is the 64-bit big-endian binary encoding of the original message length ℓ . The padded message is therefore

$$M \parallel 0^z \parallel \text{Len}_{1024}(\ell).$$

The value z satisfies $0 \leq z < 1024$. If the original message length is already a multiple of 1024 bits, then $z = 0$ and only the final length block is appended.

The final 1024-bit length block makes the encoding of the padded input injective for all supported message lengths, even though the padding before it consists only of zero bits. In particular, two supported messages that differ only by trailing zero bits have different 64-bit length encodings. The same padding rule is used by JuziHash-512 and JuziHash-1024.

No separate finalization domain constant is used in this basic specification. Domain separation between the two algorithm instances is provided by their different initialization values in Sect. 4.2. The final 1024-bit length block is part of the mandatory padding rule and is therefore unambiguously the last block processed by the hash algorithm.

4.2 Setting the initial state $H^{(0)}$

The initial 2048-bit state $H^{(0)}$ depends on the message digest size. It is generated by applying the corresponding update function once to a variant-dependent fixed value $H^{(-1)}$ and an all-zero 1024-bit block $M^{(0)}$.

For JuzziHash-512, $H^{(-1)}$ is a 2048-bit value. Its first 16 bits encode the message digest size in big-endian form, and all remaining bits are set to zero:

$$H^{(-1),0} \parallel H^{(-1),1} \parallel \dots \parallel H^{(-1),15} = 0x0200,$$

$$H^{(-1),j} = 0, \quad 16 \leq j < 2048.$$

For JuzziHash-1024, $H^{(-1)}$ is also a 2048-bit value. Its first 16 bits encode the message digest size in big-endian form, and all remaining bits are set to zero:

$$H^{(-1),0} \parallel H^{(-1),1} \parallel \dots \parallel H^{(-1),15} = 0x0400,$$

$$H^{(-1),j} = 0, \quad 16 \leq j < 2048.$$

The different initialization values separate the two algorithm instances. They are the only domain-separation values used in the basic specification. The initialization message block is

$$M^{(0)} = 0^{1024}.$$

Then

$$H^{(0)} = \begin{cases} f_{\text{sp}}(H^{(-1)}, M^{(0)}), & \text{for JuzziHash-512,} \\ f_{\text{DM}}(H^{(-1)}, M^{(0)}), & \text{for JuzziHash-1024.} \end{cases}$$

The resulting 2048-bit constants $H^{(0)}$ for JuzziHash-512 and JuzziHash-1024 are listed in Appendix B.

4.3 Computing the final state $H^{(N)}$

After padding, the message is divided into N 1024-bit blocks

$$M^{(1)}, M^{(2)}, \dots, M^{(N)}.$$

The last block $M^{(N)}$ is always $\text{Len}_{1024}(\ell)$, namely the 64-bit original message length encoded inside a 1024-bit block. Starting from $H^{(0)}$, the blocks are processed iteratively as

$$H^{(i)} = \begin{cases} f_{\text{sp}}(H^{(i-1)}, M^{(i)}), & \text{for JuzziHash-512,} \\ f_{\text{DM}}(H^{(i-1)}, M^{(i)}), & \text{for JuzziHash-1024,} \end{cases} \quad 1 \leq i \leq N.$$

For both variants, each $H^{(i)}$ is a 2048-bit internal state. After the final length block $M^{(N)}$ has been processed, $H^{(N)}$ is used to generate the message digest.

4.4 Generating the message digest

The message digest is generated from the final 2048-bit state $H^{(N)}$. The output rule depends on the selected JuzziHash variant.

4.4.1 JuziHash-512

For JuziHash-512, the final value $H^{(N)}$ is a 2048-bit sponge state. Write it as eight 256-bit state words

$$H^{(N)} = (S_0, S_1, \dots, S_7).$$

The 512-bit message digest is

$$\text{Digest}_{512}(M) = S_0 || S_1 = \text{Tr}_{512}^{\text{sp}}(H^{(N)}).$$

Thus, JuziHash-512 outputs the first 512 bits of the sponge rate part.

4.4.2 JuziHash-1024

For JuziHash-1024, the final value $H^{(N)}$ is also a 2048-bit state. The 1024-bit message digest is the capacity projection of this state:

$$\text{Digest}_{1024}(M) = \text{Cap}_{1024}(H^{(N)}).$$

The rate projection of $H^{(N)}$ is not published. It remains hidden and is needed to continue the JuziHash-1024 iteration on additional blocks. This state-hiding output rule is the main reason why JuziHash-1024 resists the ordinary length-extension attack.

5 Security Analysis of JuziHash

This section discusses the security of the two JuziHash variants. The first part gives mode-level arguments under the ideal-permutation heuristic, including collision, preimage, second-preimage and extension security. The second part summarizes the differential evidence for the concrete 24-round permutation. The section also describes statistical randomness testing and cache-timing resistance of the implementation approach. These aspects are complementary: the mode-level discussion explains what should be expected if $P_{2048}^{(24)}$ behaves like a strong public permutation, while the permutation-level analysis studies whether the concrete round function provides enough diffusion and nonlinear activity.

5.1 JuziHash-512 sponge mode

JuziHash-512 uses the sponge update function f_{sp} defined in Sect. 3.5. The 2048-bit state is divided into a 1024-bit rate part and a 1024-bit capacity part. Each 1024-bit message block is xored into the rate part, and the full 2048-bit permutation is then applied.

Under the ideal-permutation heuristic for sponge functions, a capacity of $c = 1024$ bits gives a generic security level of approximately $2^{c/2}$ against

state-recovery and inner-state matching attacks [2]. Since the digest length of JuziHash-512 is $d = 512$ bits, the expected generic collision resistance is bounded by the birthday limit $2^{d/2} = 2^{256}$. For a fixed target digest, the expected generic preimage and second-preimage resistance are both $2^d = 2^{512}$. These bounds are consistent with the sponge capacity, because $c/2 = 512$.

5.2 JuziHash-1024 full-state Davies–Meyer-like mode

JuziHash-1024 uses the full-state update function

$$H^{(i)} = f_{\text{DM}}(H^{(i-1)}, M^{(i)}),$$

where $H^{(i)}$ is always a 2048-bit state. For

$$Y = P_{2048}^{(24)}(H^{(i-1)} \oplus \text{In}(M^{(i)})),$$

the update rule is

$$\text{Rate}_{1024}(H^{(i)}) = \text{Rate}_{1024}(Y),$$

$$\text{Cap}_{1024}(H^{(i)}) = \text{Cap}_{1024}(H^{(i-1)}) \oplus \text{Cap}_{1024}(Y).$$

The 1024-bit digest is

$$\text{Digest}_{1024}(M) = \text{Cap}_{1024}(H^{(N)}).$$

Thus, the mode uses a 2048-bit internal state but publishes only the 1024-bit feed-forward branch.

For a one-block preimage step, an attacker who targets a prescribed 1024-bit output value D has to satisfy a condition of the form

$$\text{Cap}_{1024}(S) \oplus \text{Cap}_{1024}(P_{2048}^{(24)}(S \oplus \text{In}(M))) = D,$$

where the attacker may choose the 1024-bit message block M but does not control the fixed previous state in the fixed-IV hash computation. Under the ideal-permutation heuristic, this is a 1024-bit target condition. Thus for a fixed-length or ordinary-length target, a classical generic preimage attack is expected to cost about 2^{1024} operations. For very long messages, however, the interpretation of preimage-type security must take into account the number of available block positions and intermediate states. If a target computation contains about 2^k blocks, generic long-message techniques may reduce the effective cost of related preimage/second-preimage-type attacks by roughly k bits. Therefore the 2^{1024} statement for JuziHash-1024 should be read as the ordinary-message or fixed-length preimage estimate, with the long-message caveat discussed in Sect. 5.5.

Collision resistance is limited by the 1024-bit digest size. Therefore, the expected generic collision complexity is about

$$2^{1024/2} = 2^{512}.$$

This is the maximum collision level that can be claimed for any 1024-bit message digest.

For differential analysis of the update function, consider two inputs (S, M) and (S', M') . Let

$$Y = P_{2048}^{(24)}(S \oplus \ln(M)), \quad Y' = P_{2048}^{(24)}(S' \oplus \ln(M')).$$

A full internal-state collision after one update requires

$$\text{Rate}_{1024}(Y \oplus Y') = 0^{1024}$$

and

$$\text{Cap}_{1024}(Y \oplus Y') = \text{Cap}_{1024}(S \oplus S').$$

A digest-level collision at the final output only requires equality of the capacity projection of the final state, while the hidden rate projection may be different. Thus, the mode-level differential condition for the published digest is still a 1024-bit projected-output condition, but it is not caused by explicitly discarding half of the permutation output inside the compression function. The full 2048-bit permutation output is written back into the next state: the rate branch is stored directly and the capacity branch is stored after the feed-forward xor.

This distinction is useful for implementation and for analysis. The hardware does not need a separate discard stage, and an attacker who wants to continue the iteration from a digest cannot do so because the hidden rate branch of the final state is unknown. Differential analysis still has to account for the fact that only the capacity projection is published as the 1024-bit digest.

5.3 Free-start security

A free-start attack is an attack in which the adversary is allowed to choose the initial chaining state, rather than using the fixed initial state specified in Sect. 4.2. Free-start security is a stronger and different notion from the usual security of the fixed-IV hash function.

For JuziHash-512, the sponge state has 2048 bits, while only 512 bits are published as the digest. A free-start adversary who can choose arbitrary initial sponge states is no longer attacking the fixed-IV hash function. Such attacks are therefore not counted as breaks of the standard JuziHash-512 mode. The relevant generic bounds for the fixed-IV sponge mode are those in Sect. 5.1.

For JuziHash-1024, the full-state Davies–Meyer-like update still has easy free-start fixed points. Choose any 1024-bit rate value R , choose a 2048-bit permutation output value Y such that

$$\text{Rate}_{1024}(Y) = R, \quad \text{Cap}_{1024}(Y) = 0^{1024},$$

and compute

$$X = (P_{2048}^{(24)})^{-1}(Y).$$

Let

$$C = \text{Cap}_{1024}(X), \quad S = R \parallel C$$

under the rate-capacity state layout, and choose the message block M so that

$$S \oplus \text{In}(M) = X.$$

This is possible because $\text{In}(M)$ changes exactly the rate positions. Then $f_{\text{DM}}(S, M) = S$, so a free-start fixed point can be constructed with one inverse call to the permutation. This observation does not break the fixed-IV hash function, because in the real hash computation the adversary cannot choose an arbitrary internal 2048-bit state. The intended claim is fixed-IV hash security under the ideal-permutation heuristic, not strong free-start compression-function security.

5.4 Extension security

A length-extension attack tries to use a known digest $\text{Digest}(M)$ as a continuation state and to compute the digest of a related extended message without recomputing the original prefix. This is possible for a plain narrow-pipe Merkle-Damgaard hash when the digest is exactly the final chaining value. JuzziHash avoids this situation by using state-hiding output rules for both submitted instances.

The padding rule of Sect. 4.1 appends a final 1024-bit block containing the 64-bit encoding of the original message length. This makes the padded block sequence prefix-free with respect to messages that differ only by trailing zero bits. Padding alone, however, is not the main source of extension security: the essential property is that the public digest is not a full internal state from which the next compression call can be evaluated.

For JuzziHash-512, the mode is sponge-based. The published digest contains only 512 bits of the final 2048-bit state. Therefore, knowing $\text{Digest}_{512}(M)$ does not give the attacker the full internal state needed to continue the sponge computation. A direct extension attack would require recovering a compatible 2048-bit final state, and is bounded by the generic sponge security level discussed above.

For JuzziHash-1024, the internal state also has 2048 bits, while the digest publishes only the 1024-bit capacity projection:

$$\text{Digest}_{1024}(M) = \text{Cap}_{1024}(H^{(N)}).$$

To process an additional block, the attacker would need the full state $H^{(N)}$, including the hidden rate projection $\text{Rate}_{1024}(H^{(N)})$. The digest alone is therefore not a valid starting state for another compression call. Under

the ideal-permutation heuristic, recovering a compatible hidden rate branch costs about 2^{1024} work.

No extra finalization domain constant is required for this extension security argument. The protection comes from the hidden part of the final 2048-bit state. The two submitted digest sizes are already separated by different initialization values. Nevertheless, any future additional JuzziHash mode should use a distinct initialization value or another clear domain-separation method.

5.5 Second-preimage and long-message preimage-type security

Second-preimage resistance is different from collision resistance. In a collision attack, the adversary may choose both messages. In a second-preimage attack, a target message M is fixed first, and the adversary must find a different message $M' \neq M$ with the same digest. For a random d -bit hash function and an ordinary-length target message, this normally costs about 2^d work, not $2^{d/2}$ work.

For JuzziHash-512, the digest length is 512 bits and the sponge capacity is 1024 bits. Therefore, for a fixed ordinary target digest, the expected generic second-preimage resistance is about

$$2^{512}.$$

This is the same level as the preimage resistance of JuzziHash-512 and is consistent with the generic sponge security level $2^{c/2} = 2^{512}$.

For JuzziHash-1024, the digest length is 1024 bits and the internal state has 2048 bits. For fixed-length and ordinary-length messages, the expected generic preimage and second-preimage resistance are about

$$2^{1024}.$$

The adversary must hit the same final 1024-bit capacity projection as the target digest, while the hidden 1024-bit rate branch prevents the published digest from being used as a complete continuation state.

Long messages require a more careful statement. Classical long-message attacks, including the Kelsey–Schneier second-preimage technique for Merkle–Damgaard hashing, exploit the large number of intermediate states available in a target computation of many blocks [13]. JuzziHash-1024 is a wide-state mode rather than a narrow-pipe Merkle–Damgaard mode, so the published digest is not itself a continuation state. Nevertheless, for a very long target message containing about 2^k blocks, the many internal positions give a generic multi-target advantage in preimage/second-preimage-type settings. A conservative way to express the caveat is that the ordinary

2^{1024} estimate may be reduced by roughly k bits, leading to a long-message scale around

$$2^{1024-k}$$

for attacks that can exploit 2^k target positions or equivalent internal matching opportunities.

Thus, the entries in Table 2 should be interpreted as ordinary-message or fixed-length security levels. For very long target messages, the security analysis should also report the corresponding long-message estimate. Since the supported message length is less than 2^{64} bits and the message block size is 1024 bits, a valid JuziHash input has at most $2^{54} + 1$ processed blocks after padding. Therefore the largest value of k for an actual supported input is about 54, giving a worst-case long-message scale around 2^{970} for JuziHash-1024. This qualification does not change the digest-size birthday bound for collisions, but it makes the preimage and second-preimage statements more precise for extremely long inputs.

5.6 Herding attacks

Herding attacks, also known as Nostradamus attacks, are generic attacks on iterative hash functions that exploit the ability to build a diamond structure of intermediate chaining values and later connect a chosen prefix to the committed digest [12]. Their cost is closely related to the generic collision cost of the chaining value or of the published digest.

For JuziHash-512, the internal sponge state is 2048 bits, but the published digest is 512 bits. A herding or commitment-style attack at the published-digest level is therefore limited by the 512-bit output length, so its generic scale is about 2^{256} . Building a diamond on full internal sponge states would be much more expensive, because the digest does not reveal the full state.

For JuziHash-1024, the internal state is also 2048 bits, while the published digest has 1024 bits. A digest-level herding attack is therefore expected to have the same scale as a birthday attack on a 1024-bit digest, namely about 2^{512} work and large memory. A diamond on full internal states would require a much larger 2048-bit state-collision effort. Moreover, after a digest has been published, the hidden rate half of the final state prevents the adversary from simply continuing the hash computation from that digest.

A herding attack should not be confused with a standard second-preimage attack. Herding lets an adversary commit to a digest before choosing the final meaningful message prefix. It does not directly produce a second preimage for an arbitrary target message chosen by someone else. Therefore, herding does not reduce the ordinary second-preimage estimates in Table 2, but it is relevant for commitment-style applications and should be mentioned as part of the generic security analysis.

5.7 Differential analysis of the 24-round permutation

The mode-level arguments above assume that the concrete permutation does not provide exploitable high-probability differentials. This subsection summarizes the current differential evidence for $P_{2048}^{(24)}$.

The S-box used in JuziHash is an 8-bit permutation obtained from three Feistel rounds using the 4-bit S-box of Table 1. Its maximum differential probability is

$$\max_{\Delta x \neq 0, \Delta y} \Pr_x[S(x) \oplus S(x \oplus \Delta x) = \Delta y] = 2^{-4}.$$

Therefore, if a differential trail activates A S-boxes, the probability of the trail is at most

$$2^{-4A}.$$

Using a MILP/Gurobi activity model for the byte-oriented SPN structure, including the branch-number constraints of the AES MixColumns MDS layer, we found that every 8-round differential trail has at least

$$A_8 \geq 200$$

active S-boxes. The optimization is inexpensive for the current model: the minimum-activity search completed in less than two minutes on a computer with 24 cores running at 2.4 GHz per core. Since 24 rounds consist of three disjoint 8-round windows, this gives

$$A_{24} \geq 3A_8 \geq 600.$$

The probability of any single 24-round differential trail is therefore bounded by

$$2^{-4 \cdot 600} = 2^{-2400}.$$

A differential characteristic may contain many trails. To estimate the possible clustering effect, we used a conservative local multiplicity score for MDS-line support choices. In this score, an active MDS line with two active input differences contributes at most 4 support choices, an active MDS line with three active input differences contributes at most

$$\binom{4}{2} = 6$$

support choices, and an active MDS line with four active input differences contributes at most 4 support choices. To estimate clustering in the most dangerous minimum-activity case, we then fixed the 8-round activity to the minimum value

$$A_8 = 200$$

and examined the corresponding Gurobi solution pool. Under this equality constraint, the largest score observed was approximately

$$\log_2 S \leq 130.3.$$

This value should be interpreted as a loose local branching estimate for minimum-activity 8-round patterns, not as an exact count of all trails in a differential characteristic.

If this conservative 8-round multiplicity estimate is applied independently to three 8-round windows, the total trail-clustering factor is at most about

$$2^{3 \cdot 130.3} = 2^{390.9}.$$

Combining this with the single-trail bound gives the rough 24-round characteristic estimate

$$2^{390.9} \cdot 2^{-2400} = 2^{-2009.1}.$$

This is close to, but slightly weaker than, a full-state ideal-permutation level of 2^{-2048} . For this reason, the estimate should not be stated as a formal proof that every 24-round differential characteristic has probability at most 2^{-2048} . It does, however, indicate that even a very conservative clustering estimate remains far below the 2^{-1024} scale associated with a random 1024-bit projected digest-output condition in the JuziHash-1024 mode, and far below the generic collision levels of the two hash variants.

The SuperSbox concept used in the analysis of AES is also relevant as a warning about trail clustering. A SuperSbox groups several S-boxes and linear layers into a larger nonlinear component, making it possible to count many low-level trails as one higher-level differential pattern. In JuziHash, SuperSbox-like clustering can occur locally because the linear layer uses the AES MixColumns MDS matrix. This does not invalidate the active-S-box lower bound: the MILP model counts byte activity through all MDS constraints, and the lower bound $A_8 \geq 200$ still applies to every 8-round trail. The SuperSbox effect is instead a reason to consider multiple trails per characteristic. The multiplicity score above is meant to give a conservative estimate of this effect.

Moreover, JuziHash does not repeatedly apply MixColumns to one fixed AES state column structure. The permutation operates on a 256-byte four-dimensional state, and the byte-permutation layer moves bytes across different positions between rounds. As a result, local SuperSbox-like clusters are spread over the larger state as the number of rounds increases. Any full 24-round differential still has to pass through three 8-round windows with at least 600 active S-boxes in total. A tighter future analysis could formulate the MILP model directly at the SuperSbox level, but the current active-S-box and multiplicity estimates indicate a large security margin against differential attacks.

5.8 Statistical randomness tests

Statistical randomness testing is used as a practical sanity check for the output behavior of each JuziHash instance. It is not a substitute for cryptanalysis: NIST SP 800-22 notes that no statistical test suite can certify that a primitive is cryptographically secure [19]. Nevertheless, it is useful for detecting implementation mistakes, poor diffusion, biased output bits and other simple defects.

The implementation codes of JuziHash-512 and JuziHash-1024 were tested with the NIST SP 800-22 Statistical Test Suite. The NIST CSRC web page provides both the SP 800-22 Rev. 1a document and the source code of the NIST Statistical Test Suite [20]. Digest streams were generated from random messages, counter messages, variable-length messages and structured low-Hamming-weight messages. For JuziHash-1024, the full 1024-bit digest and its two 512-bit halves were also tested separately.

All tested JuziHash output streams passed the applicable NIST SP 800-22 statistical randomness tests at the usual significance level $\alpha = 0.01$. These results support the practical diffusion behavior of the submitted implementations. They do not replace the differential, preimage, second-preimage and extension-security analyses given above.

5.9 Resisting cache-timing attacks

Cache-timing attacks exploit secret-dependent memory-access patterns, such as table lookups indexed by secret state bytes. They are side-channel attacks rather than black-box collision or preimage attacks, but they are important when a hash function is used inside a keyed construction, for example in a MAC, KDF, password-hashing pre-processing step or randomized protocol transcript that contains secret data. Classical cache-timing attacks on table-based AES implementations show that small lookup tables can leak information through the cache hierarchy when the table index depends on secret data [27, 28].

JuziHash is designed so that its round function can be implemented without secret-dependent memory accesses. The byte permutation and the MDS layer are fixed linear operations: they use predetermined byte positions and xor operations, and therefore their memory-access pattern is independent of the message and of the internal state. The only nonlinear component is the 8-bit S-box. Although the S-box may be described by a table for specification and test-vector generation, it is obtained from three Feistel rounds using a 4-bit S-box. This structure allows an efficient constant-time implementation: the 4-bit S-box can be evaluated by a Boolean or bitsliced circuit, or by SIMD register-shuffle operations with the whole 16-entry table already held in registers. In both cases, the implementation does not access memory at addresses determined by secret state bytes.

For software implementations intended for side-channel-resistant use, the recommended rule is therefore:

- do not implement the 8-bit S-box by indexing a memory table with the state byte;
- use a bitsliced Boolean circuit for the 4-bit S-box, or a SIMD register-resident 4-bit shuffle, to build the three-round Feistel S-box;
- implement the MDS layer, byte permutation and round-constant addition using fixed instruction sequences and fixed memory addresses;
- keep the JuziHash-512 and JuziHash-1024 mode selection public, for example through different public API entry points and different initialization values.

Under these implementation rules, the sequence of loads, stores and cache lines touched by JuziHash is independent of the message contents and of the internal state. Consequently, the algorithm can be implemented in a cache-timing-resistant way on mainstream 64-bit processors. This statement is an implementation-level side-channel claim: it depends on using the constant-time S-box implementation described above, not merely on the mathematical definition of the permutation.

5.10 Expected generic security levels

The expected generic security levels of the two JuziHash variants are summarized in Table 2. These are mode-level claims under the ideal-permutation heuristic, supported by the permutation-level differential evidence discussed above. They do not replace a full proof for the concrete permutation.

Table 2: Expected generic security levels of JuziHash.

Variant	Digest size	State	Collision	Preimage	2nd preimage
JuziHash-512	512 bits	2048 bits	2^{256}	2^{512}	2^{512}
JuziHash-1024	1024 bits	2048 bits	2^{512}	2^{1024}	2^{1024}

The 1024-bit digest of JuziHash-1024 can provide at most 2^{512} generic collision resistance because of the birthday bound. The goal of the full-state Davies–Meyer-like mode is to retain the natural 2^{1024} generic preimage and second-preimage level for a 1024-bit digest in the fixed-length or ordinary-message setting, while also hiding half of the final 2048-bit state so that the published digest cannot be used for ordinary length extension. For very long messages, the long-message caveat of Sect. 5.5 should be applied; a target with about 2^k blocks may reduce the effective preimage-type or second-preimage-type cost by roughly k bits. Under the 64-bit message length limit, k is at most about 54 for valid inputs.

6 Performance of JuziHash

JuziHash is designed around one 2048-bit permutation and a 1024-bit message rate. Each message block invokes the 24-round permutation once. Therefore, for long messages, the main cost is the cost of the permutation per 128-byte message block; the padding and final 1024-bit length block have negligible amortized cost. The two main instances share the same message-injection interface and the same permutation implementation. JuziHash-512 uses the sponge update, while JuziHash-1024 uses the full-state Davies–Meyer-like update, which adds a 1024-bit feed-forward xor on the capacity branch. This extra xor is small compared with the cost of the 24-round permutation.

The dominant operations in software are the byte S-box layer, the length-4 MDS transformations, byte permutations and xors with round constants. These operations are well suited to SIMD implementation. In particular, the state has 256 bytes and can be stored in eight 256-bit AVX2 registers. The MDS and byte-permutation layers are regular, and the same optimized permutation code can be used by both digest-size instances. A bitsliced or register-shuffle implementation of the S-box also avoids secret-indexed cache-table lookups, as discussed in Sect. 5.9.

JuziHash also has a small memory footprint. The reference implementation only needs the 256-byte chaining state, one 128-byte message block, a small number of temporary variables, and the fixed round constants. The optimized AVX2 implementation keeps most of the working state in vector registers. No large lookup table such as a T-table is required; the nonlinear layer is implemented from the small 4-bit S-box by Boolean or in-register shuffle operations. Hence both the code size and RAM requirement are small, which is useful for embedded software, hardware-oriented implementations, and possible instruction-level acceleration.

The software performance tests in this section were carried out on an Intel Xeon E5-2680 processor. The AVX2 implementations were compiled on Microsoft Visual Studio Community 2026 using the Microsoft Visual C/C++ compiler in x64 mode. The compiler options used for the performance measurements were `/O2 /W3 /arch:AVX2`. The benchmarks were built by commands of the following form:

```
cl /O2 /W3 /arch:AVX2 bench_juzihash512_avx2_cycles.c \  
    juzihash512_avx2_single.c /Fe:bench_juzihash512_avx2_cycles.exe  
  
cl /O2 /W3 /arch:AVX2 bench_juzihash1024_avx2_cycles.c \  
    juzihash1024_avx2_single.c /Fe:bench_juzihash1024_avx2_cycles.exe
```

Table 3 reports both the best and the average number of cycles per byte over repeated runs for the single-file AVX2 implementations of JuziHash-512 and JuziHash-1024.

Table 3: Measured AVX2 software performance of JuziHash on an Intel Xeon E5-2680.

Message bytes	Iterations	JuziHash-512		JuziHash-1024	
		Best c/B	Avg. c/B	Best c/B	Avg. c/B
128	524288	30.852	31.915	30.975	31.306
1024	65536	17.177	17.203	17.273	17.290
8192	8192	15.451	15.469	15.504	15.527
65536	1024	15.276	15.298	15.317	15.335

In the original benchmark logs, two different benchmark labels correspond to the same 8192-byte message length. To keep the merged table concise after removing the test-case-label column, Table 3 reports one representative 8192-byte row for each variant.

The short-message results are slower because the fixed costs of initialization, padding, loop control and the final length block are amortized over fewer message bytes. For longer inputs, the speed approaches the steady-state permutation-processing cost. On the reported processor, both AVX2 implementations are around 15–16 cycles/byte for long messages. The JuziHash-512 and JuziHash-1024 speeds are very close because they share the same 24-round permutation core and the same 1024-bit message-injection path; the additional feed-forward xor in JuziHash-1024 has only a small cost compared with one full permutation call. Further improvements are possible by replacing the current register-shuffle S-box implementation with a more optimized bitsliced S-box circuit.

7 Design Rationale

This section summarizes the main design choices behind JuziHash. The emphasis is not only efficiency, but also simplicity, low-cost differential analysis, and implementation reuse across different digest sizes.

7.1 Rationale 1: low-cost differential analyzability

The first design rationale is analyzability. JuziHash uses a byte-oriented SPN permutation instead of an ARX-style compression function or a bit-oriented permutation. Each round contains an 8-bit S-box layer, length-4 MDS layers instantiated by the AES MixColumns matrix, and byte permutations. This structure makes the propagation of byte activity explicit and allows the minimum number of active S-boxes to be modeled directly by linear constraints.

This differs from the situation for SHA-1 and SHA-2. Their operations are well suited for software, but modular additions introduce carry-dependent differential behavior. As a result, differential analysis normally

has to handle bit conditions, carry propagation, message expansion and local collision corrections. Such analysis can be powerful, but it is not a small active-S-box-counting problem. SHA-3 has a cleaner sponge mode and a large permutation, but its internal nonlinear layer is bit-oriented and its best differential analyses require Keccak-specific reasoning. JuzziHash is not claimed to be more mature than SHA-3; the intended advantage is that its differential lower bounds are easier to formulate, reproduce and check by standard MILP tools.

The active-S-box bound also explains the choice of round number. The current MILP model gives at least 200 active S-boxes in every 8-round window. Since the full permutation has 24 rounds, any differential trail must activate at least 600 S-boxes. With an S-box maximum differential probability of 2^{-4} , this gives a single-trail bound of 2^{-2400} . Even after including the conservative multi-trail multiplicity estimate of about $2^{130.3}$ per 8-round window, the resulting 24-round estimate is about $2^{-2009.1}$. This estimate is deliberately stated as evidence, not as a complete proof of the exact maximum differential characteristic probability. Its importance is that it can be obtained with low computational cost and gives a clear, reproducible explanation for the selected security margin.

7.2 Rationale 2: simple four-dimensional AES-like structure

The second design rationale is simplicity. The permutation can be viewed as a direct four-dimensional extension of a two-dimensional AES-like SPN design principle. This wording is intentional: JuzziHash does not use the AES block cipher as a component, but adopts a similar high-level pattern of byte substitution followed by small MDS diffusion. The state is arranged as a four-dimensional byte array $T[a][b][c][d]$, while the local nonlinear and linear components remain familiar and simple: an 8-bit S-box is applied independently to every byte, and the AES MixColumns MDS matrix is applied to length-4 byte lines.

In one group of four rounds, the MDS layer is applied once along each of the four state dimensions. The byte-permutation layer then changes the relative byte positions so that diffusion produced in one dimension is transferred to the other dimensions in later rounds. Thus, JuzziHash does not rely on many different round operations. It repeatedly uses the same S-box, the same 4×4 MDS matrix, and a regular dimension schedule. This makes the permutation easy to describe, easy to implement, and easy to model in an active-byte MILP analysis.

This simplicity is also useful for hardware. A hardware implementation can reuse the same S-box cells and the same MDS datapath across different rounds, with the byte-permutation layer providing the required change of diffusion direction. Hence the 2048-bit permutation is not a collection of unrelated transformations; it is a regular four-dimensional expansion of a

small AES-like design principle.

7.3 Rationale 3: shared hardware for the two main variants

The third design rationale is hardware sharing between JuziHash-512 and JuziHash-1024. JuziHash defines two main digest sizes from the same 24-round 2048-bit permutation. JuziHash-512 uses the common sponge update function f_{sp} directly and outputs a 512-bit digest. JuziHash-1024 uses a full-state Davies–Meyer-like feed-forward mode to obtain a 1024-bit digest and a natural 2^{1024} generic preimage target.

The important implementation point is that the two variants introduce the message block into the state in the same way. In both cases, the 1024-bit message block is xored into the same four 256-bit rate positions before the permutation is applied. For JuziHash-1024, the same update function f_{sp} is first applied to the 2048-bit state, and only one additional 1024-bit feed-forward xor is needed on the capacity branch before the next state is written back. Therefore, an implementation that supports both variants can share the input-xor datapath, the state layout, the round-function control and the 24-round permutation core. The extra hardware needed for JuziHash-1024 is mainly the 1024-bit feed-forward xor and the corresponding write-back selection.

This is the main reason for using the full-state Davies–Meyer-like description. The mode keeps the rate branch as part of the next internal state and xors the previous capacity branch into the new capacity branch. Thus, no explicit output-discard stage is needed inside the update function. At digest generation time only the 1024-bit capacity projection is published, which hides the rate branch and prevents ordinary length extension.

7.4 Rationale 4: shared instruction-level acceleration

The fourth design rationale is compatibility with a possible CPU instruction or cryptographic accelerator. If JuziHash is implemented as a hardware instruction, the most natural target for acceleration is the common 2048-bit permutation rather than two separate compression functions. For example, an instruction-set extension could expose the 24-round permutation $P_{2048}^{(24)}$, or a smaller round-step instruction, on a state held in a group of vector registers or in a dedicated internal hash state.

With such an instruction, both JuziHash-512 and JuziHash-1024 can share the same accelerated primitive. The common software or microcode sequence xors the message block into the rate positions, invokes the same permutation instruction, and then applies only the variant-specific final operation: truncation for JuziHash-512, or the 1024-bit feed-forward xor, write-back selection and output projection for JuziHash-1024. Therefore, CPU support does not need two unrelated hash instructions. A single

permutation-oriented instruction can serve both digest sizes and can also support domain-separated variants that reuse the same state update.

This instruction-level sharing reduces hardware duplication in the CPU, simplifies validation of the accelerated primitive, and keeps the security analysis centered on one public 24-round permutation. It also makes the software-visible interface cleaner: different hash profiles can be built from the same low-level permutation instruction while remaining separated by initialization values, the mandatory length-block padding rule and output-selection functions.

8 Conclusion

JuziHash specifies two hash functions based on the same 24-round 2048-bit permutation. JuziHash-512 uses a sponge-like mode and outputs a 512-bit digest, while JuziHash-1024 uses a full-state Davies–Meyer-like feed-forward mode and outputs a 1024-bit digest. The two variants share the same permutation and the same message-injection structure. The permutation itself is a regular four-dimensional extension of an AES-like SPN design principle rather than an invocation of AES. This simplifies the description, allows a shared hardware implementation, and makes a single permutation-oriented CPU instruction or accelerator a natural implementation target.

The main intended advantage of JuziHash is the low cost of differential security analysis. The byte-oriented SPN structure allows the minimum number of active S-boxes to be verified by a compact MILP/Gurobi model. In the current analysis, every 8-round window has at least 200 active S-boxes, leading to a large 24-round differential margin even after a conservative estimate of trail clustering. This makes the differential behavior of JuziHash more transparent and easier to reproduce than that of SHA-1, SHA-2 and SHA-3, while the mode-level analysis gives the expected generic collision, preimage and second-preimage levels for the two digest sizes.

A State Layout, Byte Permutations, and Message Embedding

This appendix gives the exact four-dimensional state layout used by the permutation $P_{2048}^{(r)}$, its equivalent eight-word implementation layout, and the exact embedding of a 1024-bit message block into the 2048-bit state. These details are separated from the main algorithm description to keep the main text concise while still making the specification unambiguous. The same message-block embedding is used by both JuziHash variants. JuziHash-512 uses it as a sponge absorption operation, while JuziHash-1024 uses it as the common input step inside the full-state Davies–Meyer-like update function.

A.1 State layout

A 2048-bit state is conceptually a four-dimensional byte array

$$T[a][b][c][d], \quad 0 \leq a, b, c, d < 4.$$

The byte $T[0][0][0][0]$ is the leftmost byte of the serialized state, and the byte index corresponding to $T[a][b][c][d]$ is

$$64a + 16b + 4c + d.$$

Equivalently, if the serialized state is written as

$$X_0 \| X_1 \| \cdots \| X_{255}, \quad X_j \in \{0, 1\}^8,$$

then

$$T[a][b][c][d] = X_{64a+16b+4c+d}.$$

For implementation, the same state is represented as eight 256-bit state words

$$S = (S_0, S_1, \dots, S_7), \quad S_i \in \{0, 1\}^{256}.$$

Each state word is represented as 32 bytes:

$$S_i = S_{i,0} \| S_{i,1} \| \cdots \| S_{i,31}, \quad S_{i,j} \in \{0, 1\}^8.$$

The byte $S_{i,0}$ is the leftmost byte of S_i , and the serialized state is

$$S_0 \| S_1 \| S_2 \| S_3 \| S_4 \| S_5 \| S_6 \| S_7.$$

The correspondence between the two representations is

$$S_{2a+b, 4c+d} = T[a][b][c][d], \quad 0 \leq a, b, c, d < 4.$$

Equivalently,

$$S_i = S_{i,0} \| \cdots \| S_{i,31} = X_{32i} \| X_{32i+1} \| \cdots \| X_{32i+31}.$$

In the AVX2 implementation, each 256-bit state word S_i is stored in one AVX2 register.

A.2 S-box layer

The S-box layer applies the 8-bit S-box S independently to every byte of the state:

$$S_{i,j} \leftarrow S(S_{i,j}), \quad 0 \leq i < 8, \quad 0 \leq j < 32.$$

A.3 MDS layer

The MDS layer is applied independently to the first four state words and to the last four state words. For each byte position j , the vector

$$(S_{0,j}, S_{1,j}, S_{2,j}, S_{3,j})^T$$

is multiplied by the MDS matrix A over $\mathbb{F}_{2^8}$. Similarly, the vector

$$(S_{4,j}, S_{5,j}, S_{6,j}, S_{7,j})^T$$

is multiplied by the same MDS matrix. This operation is performed for all $0 \leq j < 32$.

A.4 Byte-permutation layer

After the S-box layer and the MDS layer, a byte-permutation layer is applied. The byte permutation depends on the round index modulo 3. The round index starts from 0.

For rounds with index $t \equiv 0 \pmod{3}$, each 4-byte group is rotated. For $g = 0, 1, \dots, 7$, the bytes

$$(S_{i,4g}, S_{i,4g+1}, S_{i,4g+2}, S_{i,4g+3})$$

are rotated left by $i \bmod 4$ byte positions, for $0 \leq i < 8$. Thus S_0 and S_4 are unchanged, S_1 and S_5 are rotated left by one byte in each 4-byte group, S_2 and S_6 by two bytes, and S_3 and S_7 by three bytes.

For rounds with index $t \equiv 1 \pmod{3}$, each 16-byte group is rotated. For $g = 0, 1$, the bytes

$$(S_{i,16g}, S_{i,16g+1}, \dots, S_{i,16g+15})$$

are rotated left by $4(i \bmod 4)$ byte positions, for $0 \leq i < 8$. Thus S_0 and S_4 are unchanged, S_1 and S_5 are rotated left by 4 bytes in each 16-byte group, S_2 and S_6 by 8 bytes, and S_3 and S_7 by 12 bytes.

For rounds with index $t \equiv 2 \pmod{3}$, the pairs

$$(S_0, S_4), \quad (S_1, S_5), \quad (S_2, S_6), \quad (S_3, S_7)$$

are treated as 512-bit strings. More precisely, the pair (S_i, S_{i+4}) is interpreted as

$$S_i \| S_{i+4}.$$

The pair (S_0, S_4) is unchanged. The pair (S_1, S_5) is rotated left by 16 bytes, the pair (S_2, S_6) is rotated left by 32 bytes, and the pair (S_3, S_7) is rotated left by 48 bytes.

A.5 Round-constant embedding

The round constant is

$$RC(i) = r(i) \parallel 0^{1536},$$

where $r(i)$ is 512 bits. Under the state layout above, the two 256-bit halves of $r(i)$ are xored into S_0 and S_1 , respectively, and $S_2, \dots, S_7$ receive zero from the round constant.

A.6 Message and chaining-value embeddings

A 1024-bit message block is parsed as four 256-bit words

$$M = M_0 \parallel M_1 \parallel M_2 \parallel M_3, \quad M_i \in \{0, 1\}^{256}.$$

The input embedding $\text{In}(M)$ is the 2048-bit state difference that places these four words in state-word positions S_0, S_1, S_4, S_5 :

$$\text{In}(M) = (M_0, M_1, 0^{256}, 0^{256}, M_2, M_3, 0^{256}, 0^{256}).$$

Equivalently, absorbing M into a 2048-bit state S means

$$S_0 \leftarrow S_0 \oplus M_0, \quad S_1 \leftarrow S_1 \oplus M_1,$$

$$S_4 \leftarrow S_4 \oplus M_2, \quad S_5 \leftarrow S_5 \oplus M_3,$$

with all other state words unchanged. Thus, the rate part of the state is

$$S_0 \parallel S_1 \parallel S_4 \parallel S_5,$$

and the capacity part is

$$S_2 \parallel S_3 \parallel S_6 \parallel S_7.$$

For JuziHash-512, this embedding is used directly by the sponge update

$$f_{\text{sp}}(S, M) = P_{2048}^{(24)}(S \oplus \text{In}(M)).$$

For JuziHash-1024, the same embedding is used before the permutation. If

$$Y = P_{2048}^{(24)}(S \oplus \text{In}(M)),$$

then the next state is obtained by writing the rate projection of Y directly and xoring the capacity projection of Y with the previous capacity projection of S :

$$\text{Rate}_{1024}(S') = \text{Rate}_{1024}(Y),$$

$$\text{Cap}_{1024}(S') = \text{Cap}_{1024}(S) \oplus \text{Cap}_{1024}(Y).$$

Therefore, both JuziHash variants use the same message-injection hardware and the same 24-round permutation core. The difference is only the write-back rule after the permutation and the final output projection.

B Initial State Constants

This appendix gives the concrete 2048-bit initial states $H^{(0)}$ used by JuziHash-512 and JuziHash-1024. The constants are generated by the procedure in Sect. 4.2: first set the variant-dependent value $H^{(-1)}$, then process one all-zero 1024-bit initialization block $M^{(0)} = 0^{1024}$ with the corresponding update function. The values below are written in the serialized state order $S_0 \| S_1 \| \dots \| S_7$, with each line containing eight 32-bit words in big-endian hexadecimal notation.

B.1 JuziHash-512 initial state

```
da67341b c43d1f97 c385e99b ae65d59e cf1d960d dae77906 a6f4f96b 4a8d26c3
f6f3a1b2 1344279d 68f1a22f 031e7702 e503ea25 cbf5de30 45b447ac 973299c4
15b574d6 cbc33ac2 a9b87cc4 93982423 723b518e 2f48478c e904fc5c 908e99d8
20e4c645 66237178 9d26650d 2e8cf4cc 7bb83765 ec06e09c ee897edb 581804d3
f56cefad 4827374a e8b8365c 0c874289 0a131c0f 8b60a427 42f9810a c9a00813
db312254 4422ca4b 6d1376c3 c45080bd 43445c23 12d1584c 228b8b11 d4bc6d2f
efb96f8b fb01cb43 de71b14d 2bbe4332 9306ef5f 4b77a9c1 f5ecfff1 6852d7ee
ab6ff3fe 5c5b1080 fe597713 aa717cf5 d3a03322 8de22e6b a05ec0a1 0db1abf0
```

B.2 JuziHash-1024 initial state

```
66fa3243 c9c144bf dceecfd1 e789432d 4a102929 d5670e9f 1d130a72 e67dcdad
40d7b941 d1927a1d 281654e2 c4418b03 f8b7d044 b896e530 5a383e09 e22a4435
049e741c 2a293839 74645e2d 1c1d18bb 7b3980b4 ef511fee c630f5a6 42b38045
efefadd7 7dc32ed7 bdd6fe8c 3cfbe65d 68490d25 b61aff36 8fa4a61d eb35c1a3
9d1ca813 520fe0f3 708965bd d1a06806 20d17ba3 c76d8b98 175082f4 fe9ab41f
fb85ba3f 752cccdc 3342a7a2 f7fc5f79 c9705313 29404dd9 c79137d7 e3bdf05a
00dd6a35 33a85a30 71d042dd 98202934 34f34658 f40d9676 bf068d24 84717e22
a5a1df28 b0c28afe d04adfd0 dcc4e8d7 661ae040 920585b3 ba1fe24d 907111fc
```

C Bitsliced Implementation of the 4-bit S-box

This appendix gives a compact constant-time bitwise implementation of the 4-bit S-box S_4 used in Table 1. The input bit order is

$$x = x_0 + 2x_1 + 4x_2 + 8x_3,$$

where x_0 is the least significant bit. Similarly, the output bits y_0, y_1, y_2, y_3 satisfy

$$S_4(x) = y_0 + 2y_1 + 4y_2 + 8y_3.$$

In the following C code, the variables `x0`, `x1`, `x2`, and `x3` store the bit slices corresponding to x_0, x_1, x_2, x_3 , respectively. The code overwrites them by

the output bit slices y_0, y_1, y_2, y_3 . The type `uint64_t` is used only as an example; the same Boolean circuit can be applied to machine words, SIMD registers, or hardware wires.

```
static inline void juzi_s4_bitslice_u64(uint64_t *x0,
                                         uint64_t *x1,
                                         uint64_t *x2,
                                         uint64_t *x3)
{
    const uint64_t ALL = ~(uint64_t)0;
    uint64_t m0, m1, m2, m3, a0;

    /* Input convention: x = x0 + 2*x1 + 4*x2 + 8*x3,
       where x0 is the least significant bit. */
    m0 = *x3;
    m1 = *x2;
    m2 = *x1;
    m3 = *x0;

    m3 ^= ALL;
    a0 = m0 & m1;
    m0 ^= m3 & m2;
    m3 ^= (m1 ^ ALL) & m2;
    m1 ^= m0 & m2;
    m2 ^= (m3 ^ ALL) & m0;
    m0 ^= m1 | m3;
    m3 ^= m1 & m2;
    m2 ^= a0;
    m1 ^= a0 & m0;

    /* Output convention: S4(x) = x0 + 2*x1 + 4*x2 + 8*x3. */
    *x0 = m3;
    *x1 = m2;
    *x2 = m1;
    *x3 = m0;
}
```

The circuit contains only Boolean operations and has no data-dependent memory access. It therefore provides a natural implementation method for environments where cache-timing resistance or compact hardware area is preferred over lookup-table implementation.

References

- [1] R. Anderson, E. Biham and L. Knudsen. “SERPENT: A Flexible Block Cipher with Maximum Assurance.” The first AES candidate conference, 1998.
- [2] G. Bertoni, J. Daemen, M. Peeters and G. Van Assche, “Sponge Functions.” ECRYPT hash workshop 2007.
- [3] E. Biham, A. Shamir, “Differential Cryptanalysis of DES-like Cryptosystems.” *Advances in Cryptology – Crypto’90*, LNCS 537, pp. 2-21, Springer-Verlag, 1991.
- [4] E. Biham, R. Chen, “Near-Collisions of SHA-0.” *Advances in Cryptology – CRYPTO 2004*, pp. 290–305, Springer-Verlag, 2004.
- [5] E. Biham, R. Chen, A. Joux, P. Carribault, C. Lemuet, W. Jalby, “Collisions of SHA-0 and Reduced SHA-1.” *Advances in Cryptology – EUROCRYPT 2005*, pp. 36–57, Springer-Verlag, 2005.
- [6] A. Bogdanov, L. R. Knudsen, G. Leander, C. Paar, A. Poschmann, M. J. B. Robshaw, Y. Seurin and C. Vikkelsøe, “PRESENT: An Ultra-Lightweight Block Cipher .” *Cryptographic Hardware and Embedded Systems – CHES 2007*, pp. 450–466, Springer-Verlag, 2007.
- [7] F. Chabaud, A. Joux, “Differential Collisions in SHA-0.” *Advances in Cryptology – CRYPTO 1998*, pp. 56-71, Springer-Verlag, 1998.
- [8] J. Daeman and V. Rijmen, “AES Proposal: Rijndael.” Available on-line from NIST at <http://csrc.nist.gov/encryption/aes/rijndael/>
- [9] I. Dinur and A. Shamir, “Cube Attacks on Tweakable Black Box Polynomials.” IACR ePrint, 2008. Available at <http://eprint.iacr.org/2008/385>
- [10] H. Dobbertin, “Cryptanalysis of MD4.” *Fast Software Encryption – FSE 1996*, pp. 53–69, Springer-Verlag, 1996.
- [11] H. Feistel, “Cryptography and Computer Privacy.” Scientific American, vol.228(5), May 1973, pp 15–23.
- [12] J. Kelsey, T. Kohno, “Herding Hash Functions and the Nostradamus Attack.” *Advances in Cryptology – EUROCRYPT 2006*, pp. 183–200, Springer-Verlag, 2006.
- [13] J. Kelsey and B. Schneier, “Second Preimages on n -bit Hash Functions for Much Less than 2^n Work.” *Advances in Cryptology – EUROCRYPT 2005*, pp. 474–490, Springer-Verlag, 2005.

- [14] L. Knudsen, “Truncated and Higher Order Differentials.” *Fast Software Encryption – FSE’94*, pp. 196–211, Springer-Verlag.
- [15] M. Matsui, “Linear Cryptanalysis Method for DES Cipher.” *Advances in Cryptology – Eurocrypt’93*, LNCS 765, pp. 386–397, Springer-Verlag, 1994.
- [16] S.M. Matyas, C.H. Meyer and J. Oseas, “Generating strong one-way functions with cryptographic algorithm.” IBM Technical Disclosure Bulletin, 27 (1985), 5658–5659.
- [17] National Institute of Standards and Technology, “Secure Hash Standard (SHS),” FIPS PUB 180-4, August 2015.
- [18] M. J. Dworkin, “SHA-3 Standard: Permutation-Based Hash and Extendable-Output Functions,” FIPS PUB 202, August 2015.
- [19] A. Rukhin, J. Soto, J. Nechvatal, M. Smid, E. Barker, S. Leigh, M. Levenson, M. Vangel, D. Banks, N. Heckert, J. Dray, S. Vo and L. Bassham, “A Statistical Test Suite for Random and Pseudorandom Number Generators for Cryptographic Applications,” NIST SP 800-22 Rev. 1a, April 2010.
- [20] National Institute of Standards and Technology, “NIST SP 800-22: Download Documentation and Software,” Computer Security Resource Center. Available at <https://csrc.nist.gov/projects/random-bit-generation/documentation-and-software>.
- [21] J. Stern, S. Vaudenay. “CS-Cipher.” *Fast Software Encryption – FSE’98*, pp. 189–205, Springer-Verlag, 1998.
- [22] X. Wang, X. Lai, D. Feng, H. Chen, X. Yu, “Cryptanalysis of the Hash Functions MD4 and RIPEMD.” *Advances in Cryptology – EUROCRYPT 2005*, pp. 1–18, Springer-Verlag, 2005.
- [23] X. Wang, H. Yu, “How to Break MD5 and Other Hash Functions.” *Advances in Cryptology – EUROCRYPT 2005*, pp. 19–35, Springer-Verlag, 2005.
- [24] X. Wang, H. Yu, Y. L. Yin, “Efficient Collision Search Attacks on SHA-0.” *Advances in Cryptology – CRYPTO 2005*, pp. 1–16, Springer-Verlag, 2005.
- [25] X. Wang, Y. L. Yin, H. Yu, “Finding Collisions in the Full SHA-1.” *Advances in Cryptology – CRYPTO 2005*, pp. 17–36, Springer-Verlag, 2005.
- [26] R. Winternitz. “A secure one-way hash function built from DES.” IEEE Symposium on Information Security and Privacy, p. 88–90, 1984.

- [27] D. J. Bernstein, “Cache-timing attacks on AES,” 2005. Available at <https://cr.yp.to/antiforgery/cachetiming-20050414.pdf>.
- [28] D. A. Osvik, A. Shamir and E. Tromer, “Cache attacks and counter-measures: the case of AES,” CT-RSA 2006, Lecture Notes in Computer Science, vol. 3860, pp. 1–20, Springer, 2006.